DROIDKAIGI 2026 · TOKYO

Android Platform / Audio Techniques

Android Gets a Voice

Building Real-Time Agents

CONNECT

LinkedIn: https://www.linkedin.com/ln/mark-j-wickham

ASSETS

Github: https://www.github.com/m15-ai/droidkaigi2026

Mark Wickham

The Future is Voice …

- ✓ *Voice input is 4x faster than typing*
- ✓ *Voice carries emotion*
- ✓ *Voice is immediately recognizable*
- ✓ *Voice is simple, only need a mic and speaker*
- ✓ *Powerful LLM's can chat with us*
- ✓ *Coding agents — it's now easy to create complex software*

… and our Android devices provide the perfect platform to express it !

What We Will Cover

① Coding Approach

Coding Agents
Voice Requirements
Markdown Template

② Voice Fundamentals

Pipelines, Latency, Streaming
Streaming, Echo cancel
Connectivity, Barge-In

③ Three Voice Architectures

Local

Google Voice Pipeline
Completely on-device
No Cloud, No Network.

Cloud

API key management
WebSocket streaming
Multi-Lingual (Japanese)

Agentic

Android Pipecat Client
WebRTC streaming
“Homer” OpenClawAgent

Coding Approach

Coding Agents: Architect, Don't Code

Our job shifts from writing code to making decisions.

My process to create Voice Apps on Android

1. **Define requirements using structured template**
2. **Feed the template to your coding agent**
3. **Ask the Agent to “scaffold” the App**
4. **Review & iterate**

Android_Voice_App_Requirements_Template.md

We can realize a working app in just a few hours !

Android_Voice_App_Requirements_Template.md

Android Voice App Requirements Template

App Identity

- App name: Google Voice Pipeline
- Short app name: GVP
- Package name: com.m15.gvp
- Primary purpose: Fully on-device voice AI agent — STT, LLM, and TTS all run locally with zero cloud calls during pipeline operation
- Target user: Developer / power user evaluating on-device voice agent architectures
- Primary language: English (US)
- Supported languages: English only for v1
- Future language expansion: Yes — Sherpa-ONNX supports multilingual models; LiteRT models for other languages may appear on Hugging Face

Project Constraints

- Android Studio on: Windows 11 (Ladybug or later)
- Kotlin only: Yes
- Minimum SDK: 31 (Android 12)
- Target SDK: 35 (Android 15)
- Single Activity: Yes
- MainActivity only: Yes
- No fragments: Yes
- No XML layouts: Yes
- Jetpack Compose only: Yes
- Portrait only: Yes
- Landscape allowed: No
- Edge-to-edge UI: Yes
- Material Design 3: Yes — dynamic color on SDK 31+, static fallback otherwise, System/Light/Dark theme switching
- No BOM characters at start of files: Yes

Architecture

- Architecture pattern: MVVM

Voice Fundamentals

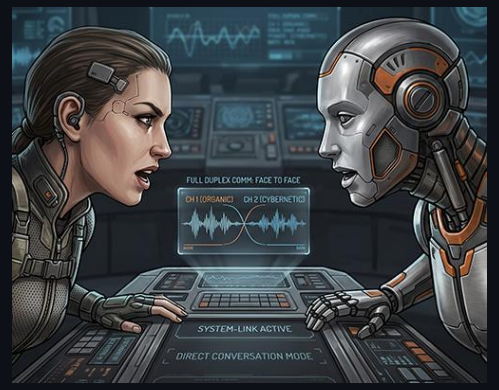
Realtime Voice — "Feels Like a Real Conversation"

Turn-Based Voice

- ✗ Tap to speak
- ✗ Wait for recognition
- ✗ Wait for LLM response
- ✗ Wait for TTS playback
- ✗ No interruption possible

The shift to REAL-TIME

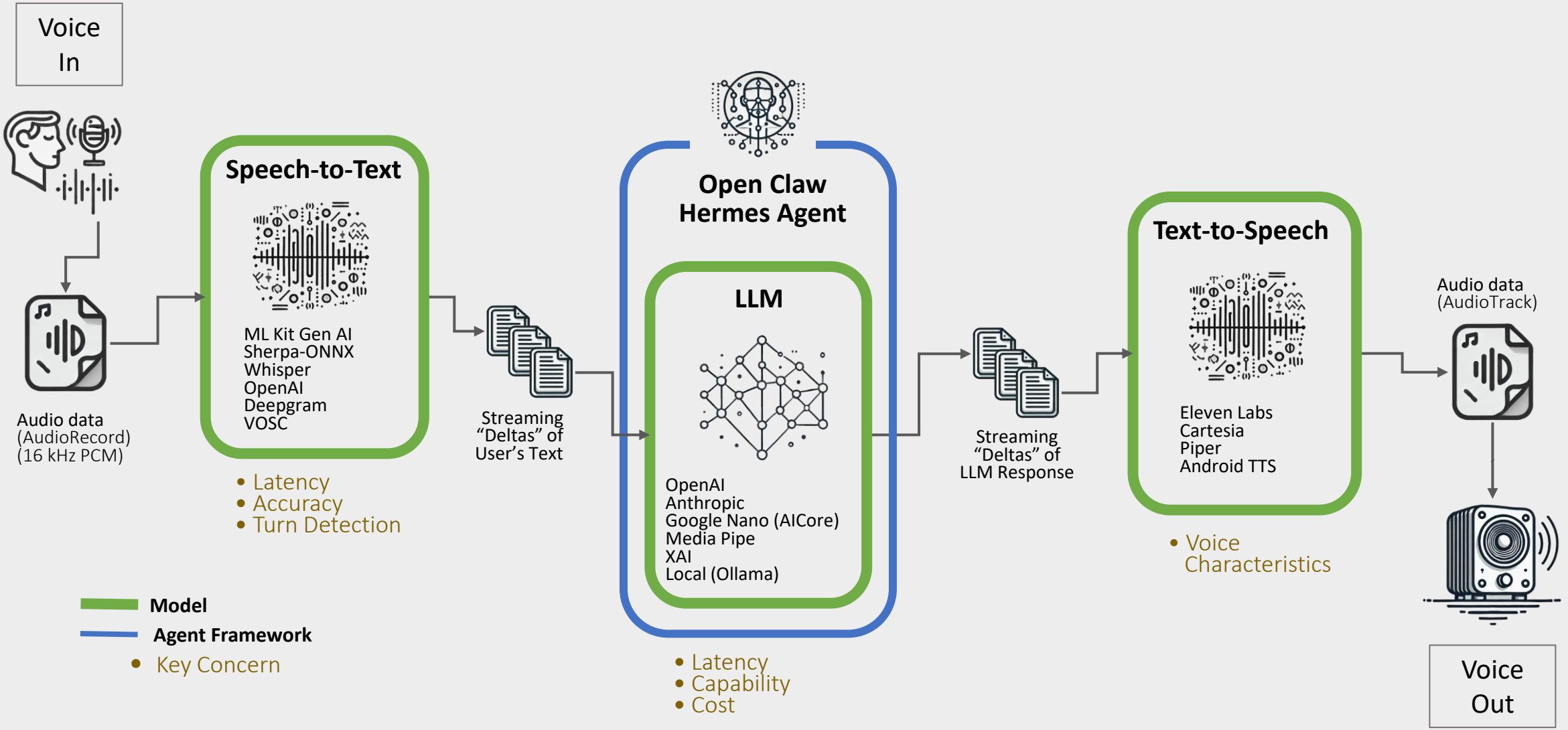
- ✓ Continuous **Streaming** — no wait
- ✓ **Full Duplex**
- ✓ Token-**Streaming LLM** responses
- ✓ **Low-latency** TTS playback
- ✓ Full **Barge-In** / interruption



Concept	Description
WebSocket	A consistent two-way connection for continuously streaming audio, transcripts, and AI responses. Cliff
WebRTC	A real-time media protocol optimized for low-latency and voice communications D
Packets	**Delta packets** are incremental updates of text or audio, streamed as the model generates it. **Final packets** are the completed stable result, such as a completed transcription or LLM response.
Turn Detection	Detecting when the user has completed speaking and the agent can process.
AEC	Automatic Echo Cancellation, Hardware implementation cancels the speaker output from mic input
Barge-in	The ability for our pipeline to detect and handle interruptions.

Conversational AI Architecture

(Realtime, Full Duplex, Streaming)



VOICE PIPELINE PERFORMANCE

Latency

Makes or breaks your voice app — the true feel of the experience.

TTFT = Time to First Token

Target for voice systems — **~1,000 ms**

Each Demo App has a Latency Class to measure this Critical Value

Stage	Time (ms)
Hardware Mic Capture	40
Opus Encoding	21
Network Stack & Transmit	10
Packet Handling	2
Jitter Buffer & Opus Decoding	41
Transcription (STT) & Endpointing	**300**
LLM TTFT	**650**
Sentence Aggregation	20
TTS TTFB	120
Opus Encoding	21
Packet Handling	2
Network Stack & Transmit	10
Jitter Buffer & Opus Decoding	41
Hardware Speaker Output	15
Total	**1293**

Source: "Voice AI and Voice Agents" — Pipecat

LLM — the Voice AI Brain

- The LLM is the perceived intelligence of your Agent.
- Most widely used models for voice agents are GPT-4.1, GPT-5.1, and Gemini 2.5 Flash.
- Avoid Reasoning models.
- Not all models are optimized for latency (Sonnet 3.7 ☹).

- It’s Always a Tradeoff

LLM	Median TTFT (ms)	Cost/ 3-Min
GPT-4.1	536	$0.32
GPT-5.1	739	$0.10
Gemini 2.0 Flash	380	$0.004
Gemini 2.5 Flash	597	$0.002
Gemini Nano		—
Llama 4 Maverick	290	—
Claude 4.5 Haiku	**637**	**$0.006**
Claude 3.7 Sonnet	1,410	
Claude 4.6 Sonnet	**850**	**$0.018**
Kimi 2.6 Cerebrus	452	
Grok-4-fast		
Qwen		
Nemotron 3 Ultra	541	

Source: "Voice AI and Voice Agents" — Pipecat

Architecture Tradeoffs

Dimension	Local (GVP)	Cloud (Cliff)	Agentic (Pipecat)
Pipeline runs	On device	Cloud providers	Pipecat server
Network	None (fully offline)	Always (WebSocket + HTTPS)	Always (P2P WebRTC)
STT	ML Kit GenAI, Sherpa-ONNX	Deepgram flux-general-multi	Deepgram Nova-3 (server)
LLM / Brain	AI Core Nano, MediaPipe(LiteRT)	Claude Sonnet 4.5/4.6	OpenClaw (GPT4-1)
TTS	Android TextToSpeech	Deepgram Aura-2	Cartesia Sonic-2
Latency	Sub-650ms (device HW)	~700-1500ms (network)	~850ms (GPU baseline) + Tools
Target	minSdk = 31	minSDK = 26	minSDK = 26
Auth model	None needed	Ephemeral tokens	None (trusted network)
Best for	Privacy, offline, zero cost	Best quality, production	Agent frameworks, tool calls, memory
Client complexity	Full pipeline on device	WebSocket + SSE streaming	Thin — mic + speaker only

Common: **UI**—Compose+Material 3, **State**—Kotlin Coroutines+Flow, MVVM, **Audio**—AudioRecord + AudioTrack, **Storage**—Room DB + SharedPrefs

The right solution depends on YOUR requirements.

Local (GVP)

Completely on-device — no cloud, no network, no problem

STT

ML Kit GenAI (expr)
Sherpa-ONNX (multi)

LLM

Google AI Core Nano
Google ML-Kit MediaPipe

TTS

Android TTS

GVP App Settings

Inference Pipeline (LLM)

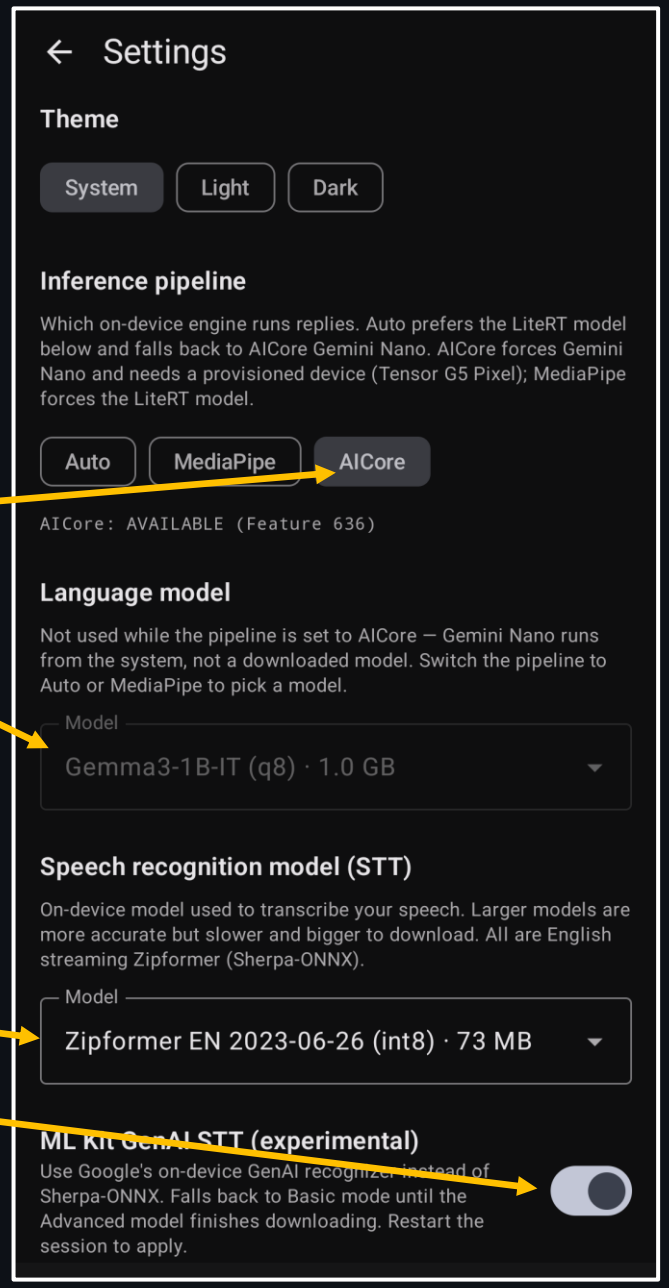
AICore for Nano (Pixel/G5)

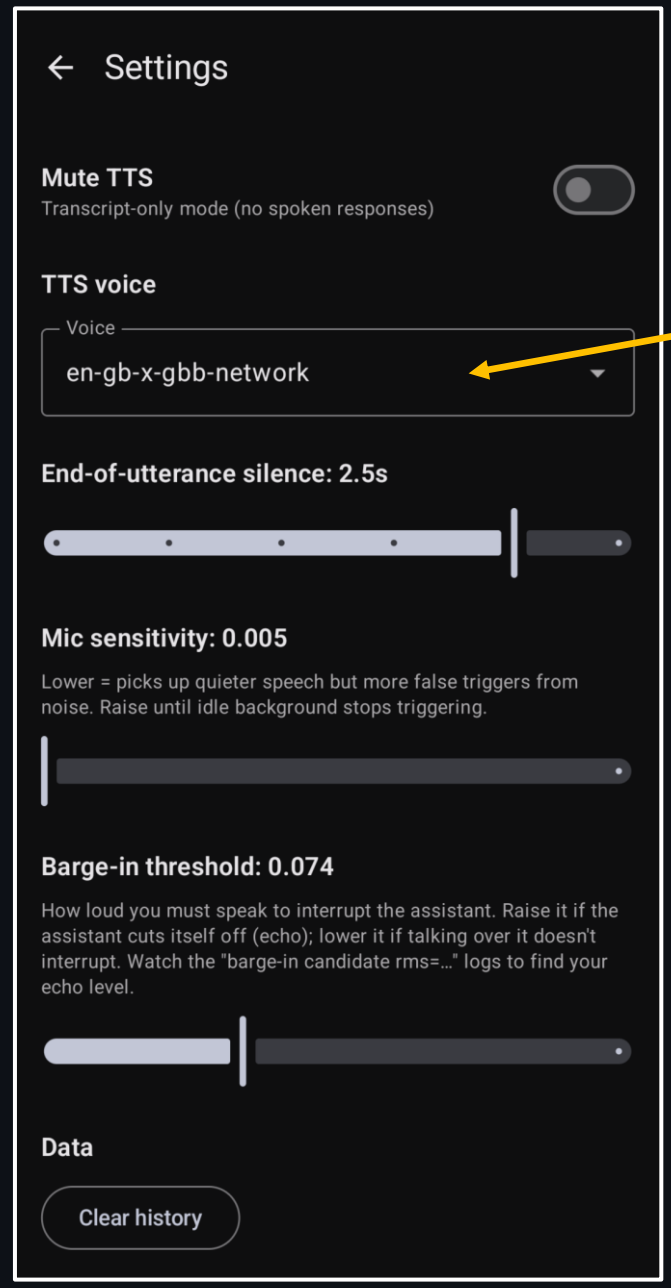
MediaPipe
Downloadable Models

Speech-to-Text

Sherpa/ONNX
Selectable Models

ML Kit GenAI (Pixel/G5)





Text-to-Speech

Android TTS
Selectable Voice

GVP App Overview

Fully Local On-Device Voice Pipeline

A complete voice loop — STT, LLM, TTS — running entirely offline on a stock Android phone.

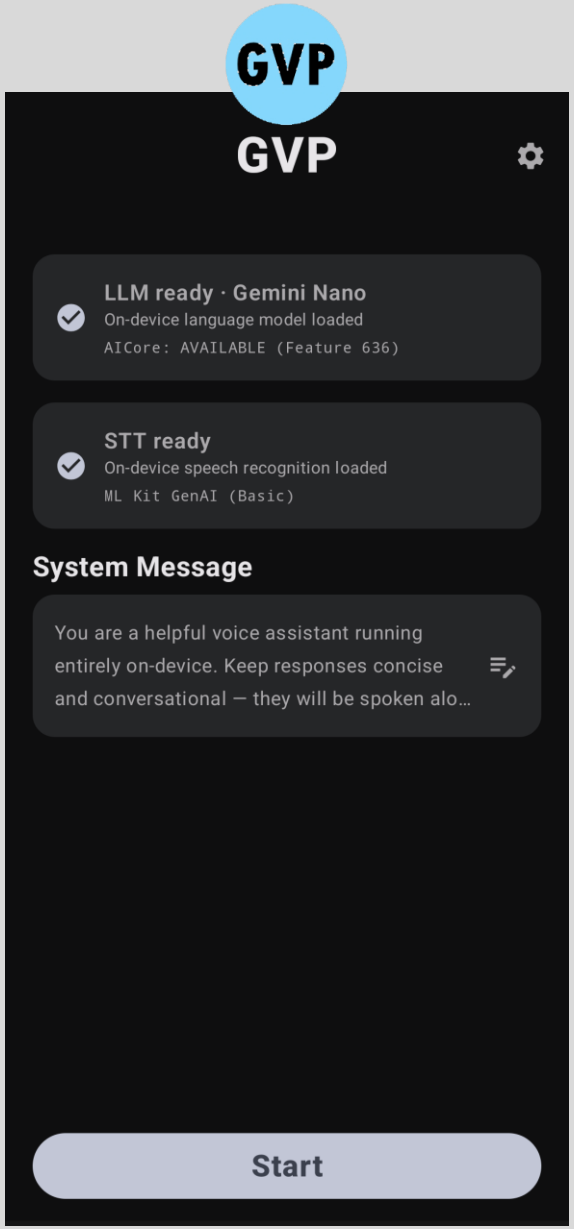
KEY CONCEPTS

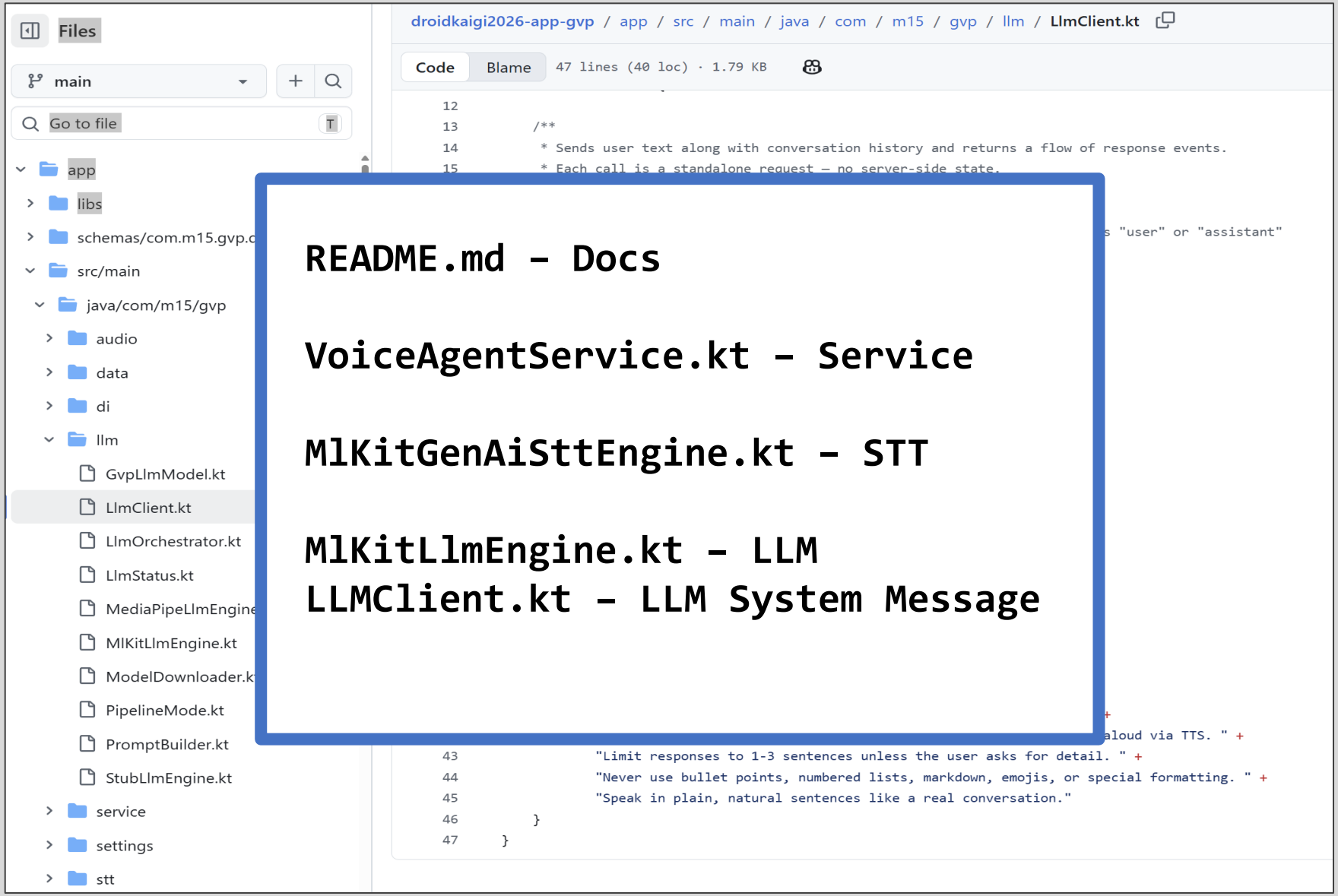
- **Zero Cloud Calls** — Complete STT → LLM → TTS loop runs entirely on-device.
- **Privacy & Offline First** — Voice and transcripts never leave the handset.
- **622 ms TTFT** — Measured time-to-first-token on Pixel 10 (Tensor G5).
- **Two On-Device LLMs** — MediaPipe/LiteRT (any device) or Gemini Nano/AICore (Pixel).
- **Model Catalog** — LLMs (Qwen 0.5B/1.5B, Phi-4-mini, Gemma 1B, Gemini Nano)
  Downloads persist. Zero API costs after model download.

THE TRADEOFFS

- **LLM Quality** — Limited by on-device model size.
- **Requires Expensive High-End Device** — GPU and Memory for model weights

GVP App Demo and Code Review





README.md – Docs

VoiceAgentService.kt – Service

MlKitGenAiSttEngine.kt – STT

MlKitLlmEngine.kt – LLM
LLMClient.kt – LLM System Message

Cloud (Cliff)

State of the Art Cloud Models

STT

Deepgram Flux
Websockets Streaming

LLM

Claude
Sonnet 4

TTS

Deepgram Aura-2
Websockets Streaming

- ✓ Security with server tokens
- ✓ Multilingual (Japanese)

Cliff App Overview

State-of-the-Art (SOTA) Cloud Pipeline

Best-in-class cloud models with full-duplex streaming.

KEY CONCEPTS

- **Full-Duplex Streaming** — STT, LLM, and TTS all stream concurrently.
- **Sub-Second TTFT** — Live TTFT readout (e.g. TTFT 842 ms).
- **Multilingual Support** — Japanese end-to-end.
- **Real Barge-In** — Interruptions cancel in-flight LLM response and cancel audio.
- **Stateless by Design** — Drives Claude’s plain Messages API, No session management.
- **No API Keys on Device** — Backend manages Deepgram and Claude tokens.
- **Compose Audio Orb** — Reactive orange orb driven by max(ttsLevel, micLevel).

Security – Managing API Keys

API Key Lookup

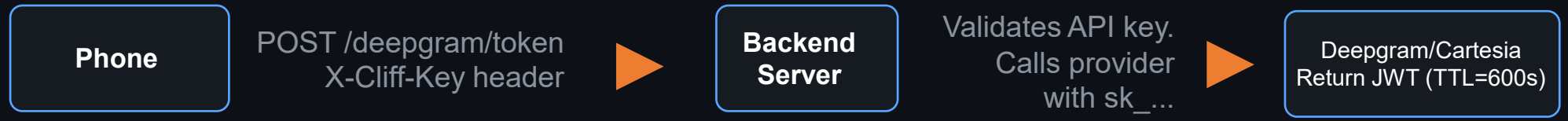
The APK never holds long-lived API keys for the speech providers.

No app update to rotate
Master keys live in the backend's env file.

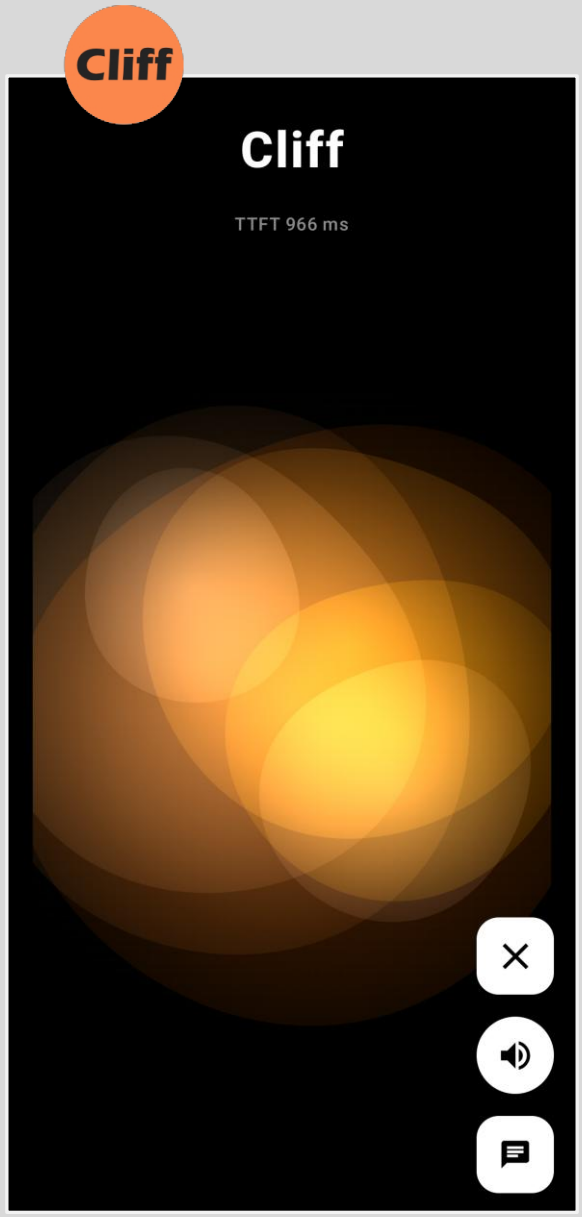
Ephemeral Tokens

Backend server mints short-lived tokens on demand. The APK holds nothing useful.

Token Lifetime - 10 mins
Expires before it's worth abusing.



Cliff App Demo and Code Review



Files

main

Go to file

- app
- schemas
- src/main
- java/com/m15/c
- audio
- data
- di
- net
- claude
- ClaudeStrear
- flux
- LlmClient.kt
- prefs
- tts
- ui
- util
- App.kt
- BargeInController.kt
- CustomPromptScreen.kt
- MainActivity.kt
- VoiceAgentViewModel.kt

droidkaigi2026-app-cliff / app / src / main / java / com / m15 / cliff / net / LlmClient.kt

Code | Blame | 63 lines (56 loc) · 2.8 KB

README.md – Docs

FluxClientImpl.kt – Deepgram Flux STT

LlmClient.kt – Claude LLM

DeepgramTtsClient.kt – Deepgram TTS

cliff_server.py – Backend

re Cliff. " +

```
55              "so they read naturally aloud, and do not mix Latin script into your replies " +
56              "unless quoting a proper noun that has no Japanese form. " +
57              "Your responses are spoken aloud via text-to-speech. " +
58              "Keep responses short and conversational — 1 to 3 sentences max. " +
59              "Never use bullet points, numbered lists, markdown, emojis, or special formatting. " +
60              "Speak in plain, natural sentences like a real conversation. " +
61              "If a topic needs more detail, offer to explain further rather than dumping everything at once."
62      }
63  }
```

Agentic Voice (Pipecat)

Voice AI is no longer just inference.
We make it more powerful by adding
orchestration, memory, and tool calling.

Agentic Voice Architecture

Android App
Pipecat Client
Capture mic audio, Play TTS, Render UI, Maintain session

→

Pipecat
Thin Transport
Open Source, Real-time audio transport. WebRTC

→

OpenClaw/Hermes
Agentic AI
on the Server
Run agents, manage state, connect tools, guard credentials, and evolve frameworks.

→

MCP
Model Context Protocol
AI apps to connect to tools and data sources.

Connecting your Android device with backend server

Tailscale Mesh Network (Recommended)

Install on any device or server
Works across Wi-Fi, cellular, and public network
Free tier covers most hobby and small-team projects

Alternatives: Self-hosted VPN (WireGuard, OpenVPN)
Tunnels (ngrok, Cloudflare Tunnel)

Agent Frameworks

- Pipecat transport is supported by both OpenClaw and Hermes.
- Easy to access tool calls, persistent memory, and skills.
- We can access multiple agents by PORT from the client.

OpenClaw

Personal AI Assistant

- Tool calls (web, files, code)
- Plugin / skills ecosystem
- Multi-modal interfaces
- Large voice community

Hermes

Self-Improving Agent

- Persistent memory, learning
- Auto-creates skills from use
- Long-running agent loops
- Self-improving over time

Pipecat demo implements a **Baseball Agent** called **“Homer”** on OpenClaw

Pipecat Client App Overview

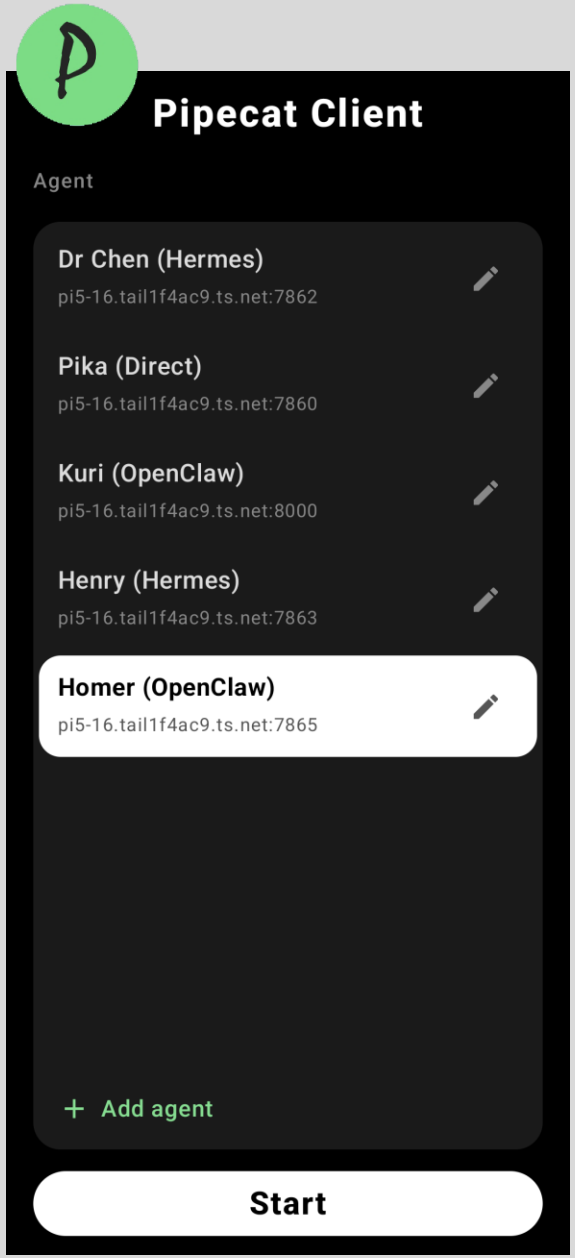
Agent Integration with Server-Side Brain

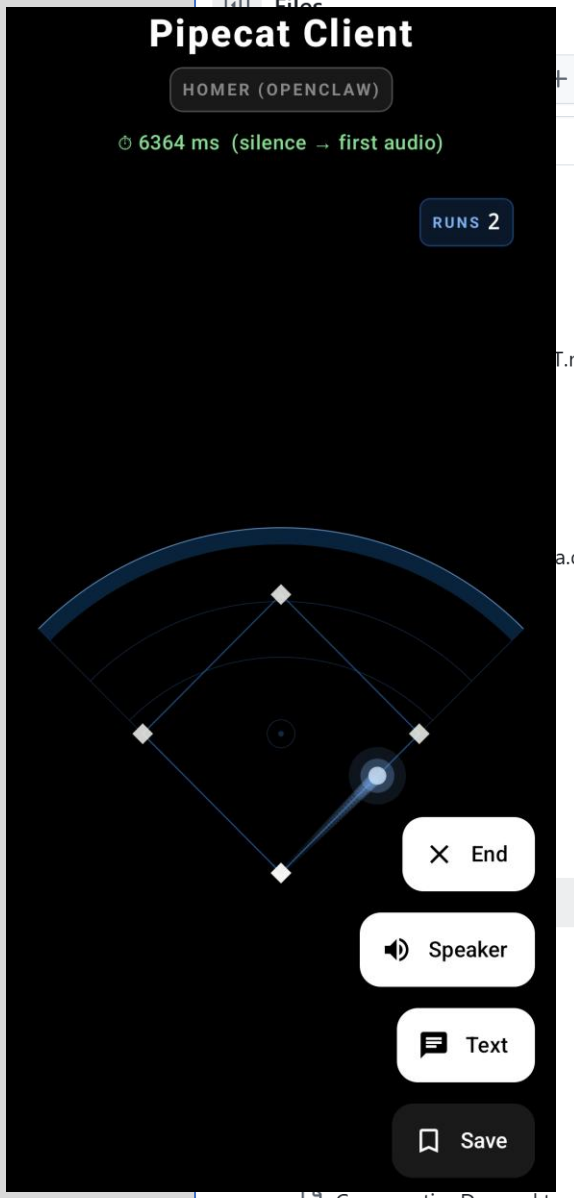
The phone is the mic and speaker. All intelligence lives on a Pipecat server.

KEY CONCEPTS

- **Phone** is just a **Mic + Speaker** —
  All intelligence (STT, LLM, TTS, turn-taking, barge-in) lives on the Pipecat server.
- **Simple Architecture** — No Token Management.
  Connect by POSTing an SDP offer to /api/offer. Pure P2P WebRTC.
- **Entire SDK in One File** — PicaVoiceClient.kt.
- **Instant Barge-In** — Stops local playback the instant you speak
- **Agent Integration** — Server-side brain, Agent framework — OpenClaw or Hermes.
  Tool calls and memory built in.
  Huge Python support for Agentic AI available on the server side.
- **AEC3 echo cancellation** — Free from libwebrtc on both ends.

Pipecat Client App Demo and Code Review





droidkaigi2026-app-pipecat / app / src / main / java / com / m15 / pica / net / PicaVoiceClient.kt

Code Blame 120 lines (109 loc) · 4.72 KB

```
33  class PicaVoiceClient(
34      context: Context,
35      private val serverUrl: String,
36      private val listener: Listener,
```

README.md – Docs

PicaVoiceClient.kt – Pipecat

BasesVisualizer.kt - Visualizer

Homer OpenClaw Server Repo:
- **Skills/mlb.py**
- **Workspace/Soul.md**
- **Workspace/Identity.md**
- **Workspace/Agents.md**

```
62          override fun onUserStoppedSpeaking() = listener.onUserStoppedSpeaking()
63          override fun onBotStartedSpeaking() = listener.onBotStartedSpeaking()
64          override fun onBotStoppedSpeaking() = listener.onBotStoppedSpeaking()
65          override fun onUserAudioLevel(level: Float) = listener.onUserAudioLevel(level)
66          override fun onRemoteAudioLevel(level: Float, participant: Participant) =
67              listener.onRemoteAudioLevel(level)
68          override fun onUserTranscript(data: Transcript) =
69              listener.onUserTranscript(data.text, data.`final`)
```

ConversationDrawer.kt

Key Takeaways

1. **Architect, Don’t Code**
   Coding agents write the code

2. **Three Architectures, Match Your Requirements**
   Local, Cloud, Agentic

3. **Ephemeral Tokens Beat Secret-Stuffing**
   No API keys in the APK

4. **Powerful Agentic Frameworks**
   Best in class open-source python — OpenClaw and Hermes

Thank You

Android Gets a Voice: Building Real-Time Agents